E-Commerce Microservices Architecture Documentation v2.4

1. System Overview

This document details the architectural constraints, API specifications, and data flow strategies for the core e-commerce backend. The system is decomposed into domain-driven microservices to ensure loose coupling and high cohesion.

Notice: All developers must adhere to the *Recursive Consistency Protocol* defined in the attached PDF (sys_arch_v2.pdf). Failure to follow the event-sourcing patterns defined below will result in transaction anomalies during high-concurrency sales events (e.g., Black Friday).

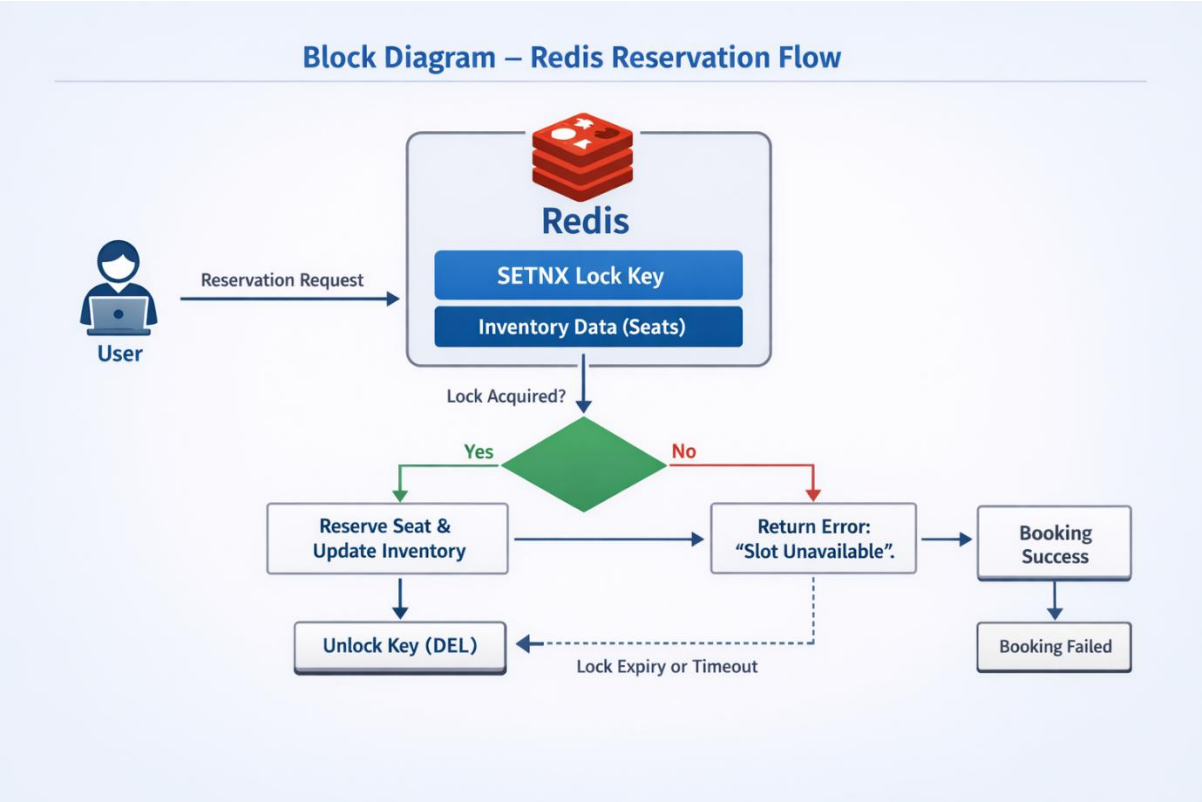
---

2. Service A: Inventory Management Service (IMS)

The IMS is responsible for tracking stock levels, reserving items during checkout, and managing warehouse synchronization.

2.1 Data Consistency Model

Unlike the User Service, the IMS utilizes a strong consistency model. When a user adds an item to their cart, a "soft reservation" is made in Redis. If the checkout is not completed within 15 minutes, the reservation expires. This prevents "inventory holding" attacks.

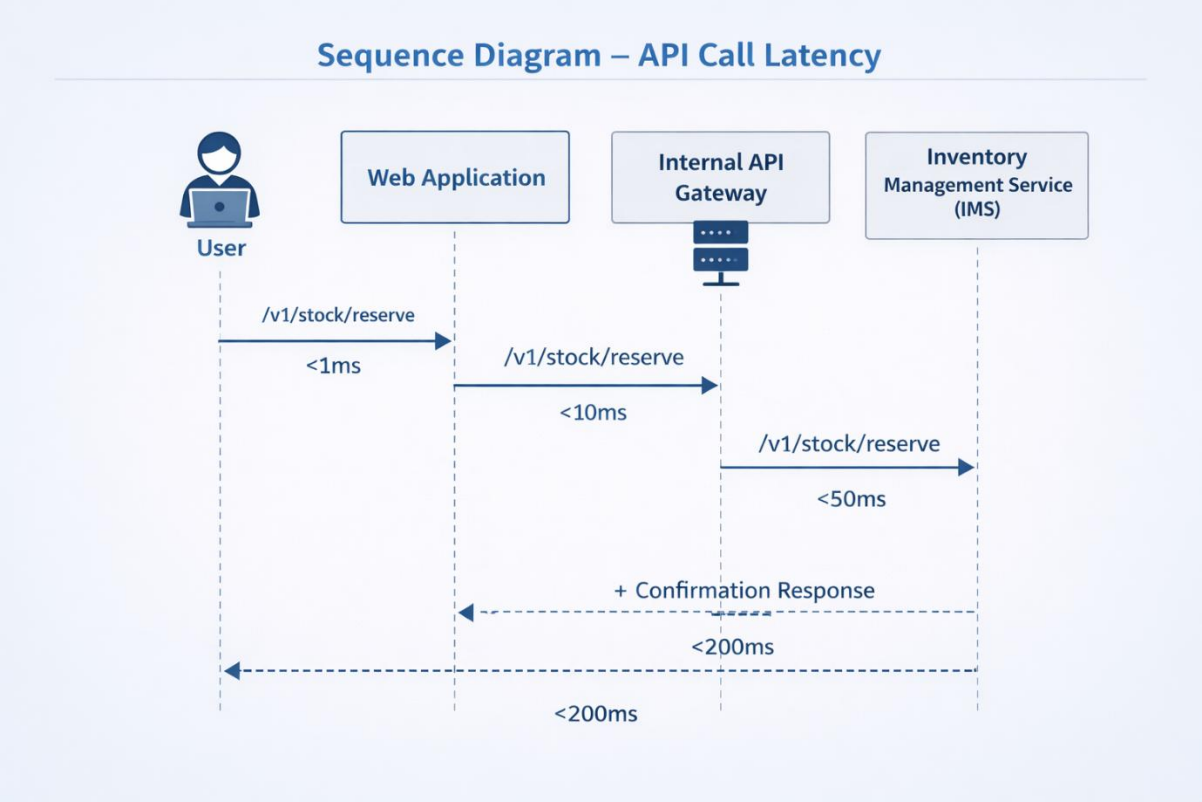


(Description: Diagram showing the handshake between the Checkout Service and Redis TTL keys for inventory locking.)

2.2 API Endpoints

The following endpoints are exposed via the internal API Gateway.

Endpoint	Method	Description	Latency SLA
/v1/stock/{sku}	GET	Retrieves current stock level. Cached for 5s.	< 20ms
/v1/stock/reserve	POST	Locks inventory for a specific session ID.	< 50ms
/v1/warehouse/sync	PUT	Bulk update from ERP systems.	N/A (Async)



2.3 Database Schema

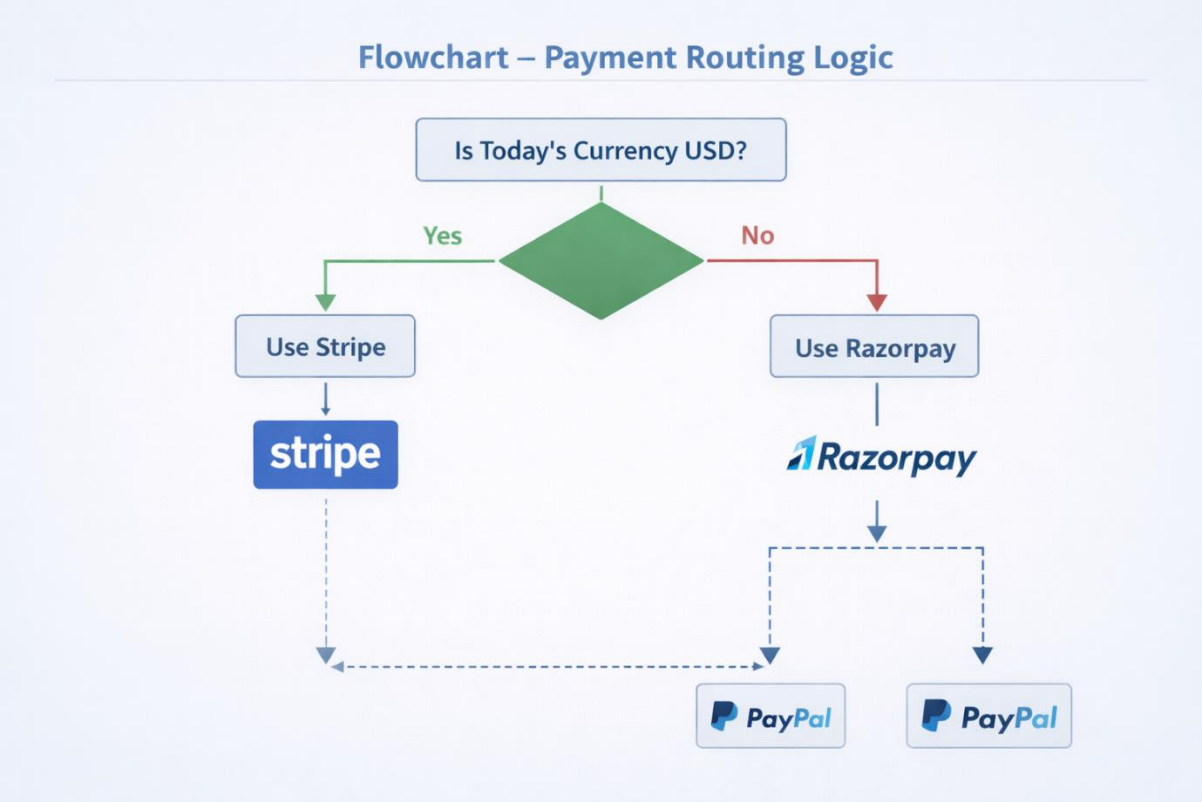
The IMS uses PostgreSQL 15 with the following core tables:

- inventory_items (PK: sku_id, FK: warehouse_id, int: quantity)
- reservations (PK: res_id, timestamp: expiry, status: ACTIVE/EXPIRED)
- audit_log (text: event_type, jsonb: payload)

---

3. Service B: Payment Gateway Aggregator (PGA)

The PGA acts as an abstraction layer over third-party providers (Stripe, PayPal, Razorpay). It handles sensitive PCI-DSS data and must never log full PAN numbers.



(Description: Logic tree deciding which provider to use based on user currency and region.)

3.1 Error Handling & Retries

Payment failures are categorized into "Soft Declines" (insufficient funds, network timeout) and "Hard Declines" (stolen card, fraud).

- **Soft Declines:** The system retries the transaction up to 3 times with exponential backoff (1s, 2s, 5s).
- **Hard Declines:** Immediate rejection signal sent to the frontend.

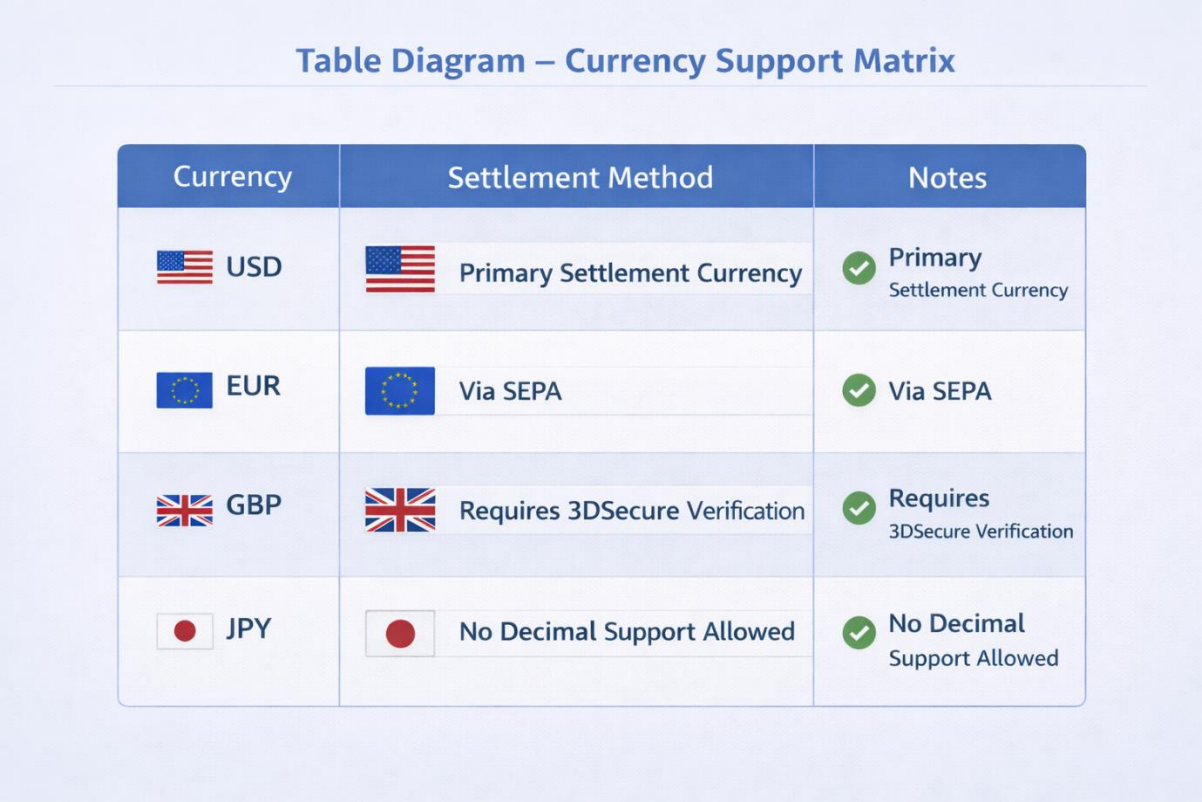
Critical Alert: Review the pci_compliance_audit_2024.pdf attachment before modifying the PaymentTokenizer class.

3.2 Supported Currencies

The PGA currently supports the following currencies with real-time conversion rates fetched from the OpenExchangeRates API:

1. **USD** (United States Dollar) - Primary settlement currency.
2. **EUR** (Euro) - Handled via SEPA where applicable.
3. **GBP** (British Pound) - Requires additional 3DSecure verification.

4. **JPY** (Japanese Yen) - No decimal support allowed.



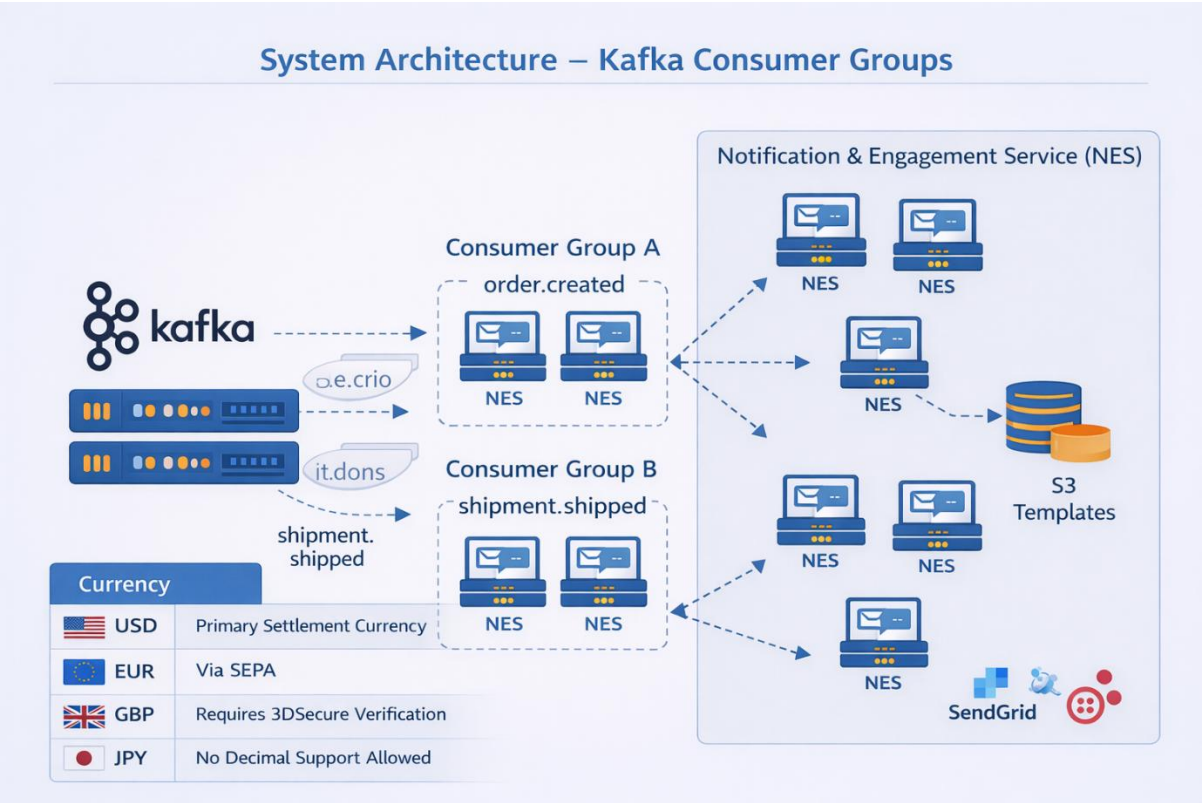
---

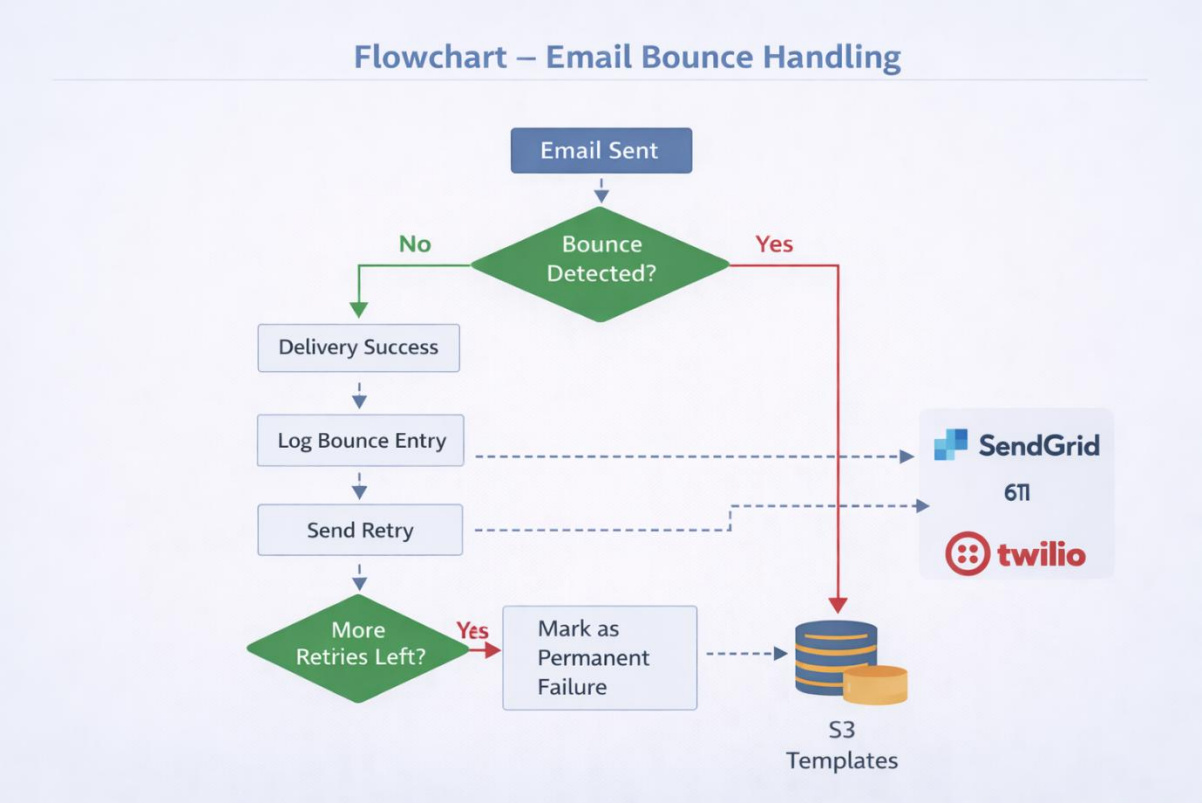
4. Service C: Notification & Engagement Service (NES)

The NES is a stateless worker service that consumes messages from Kafka topics (order.created, shipment.shipped) and dispatches emails, SMS, or push notifications.

4.1 Notification Flow

1. **Ingestion:** Kafka consumer reads the event payload.
2. **Deduping:** Redis is checked to ensure the user hasn't received this notification in the last 60 seconds.
3. **Rendering:** Jinja2 templates are populated with user data.
4. **Dispatch:** Route to SendGrid (Email) or Twilio (SMS).



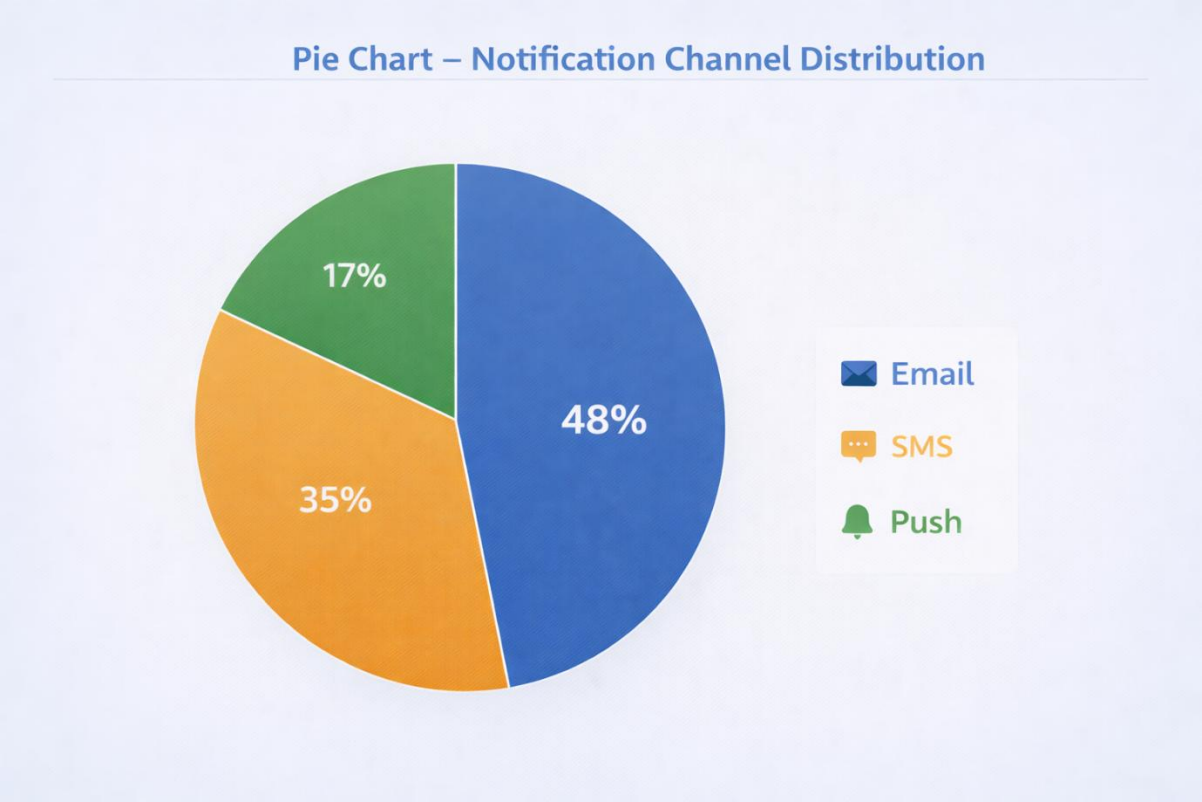


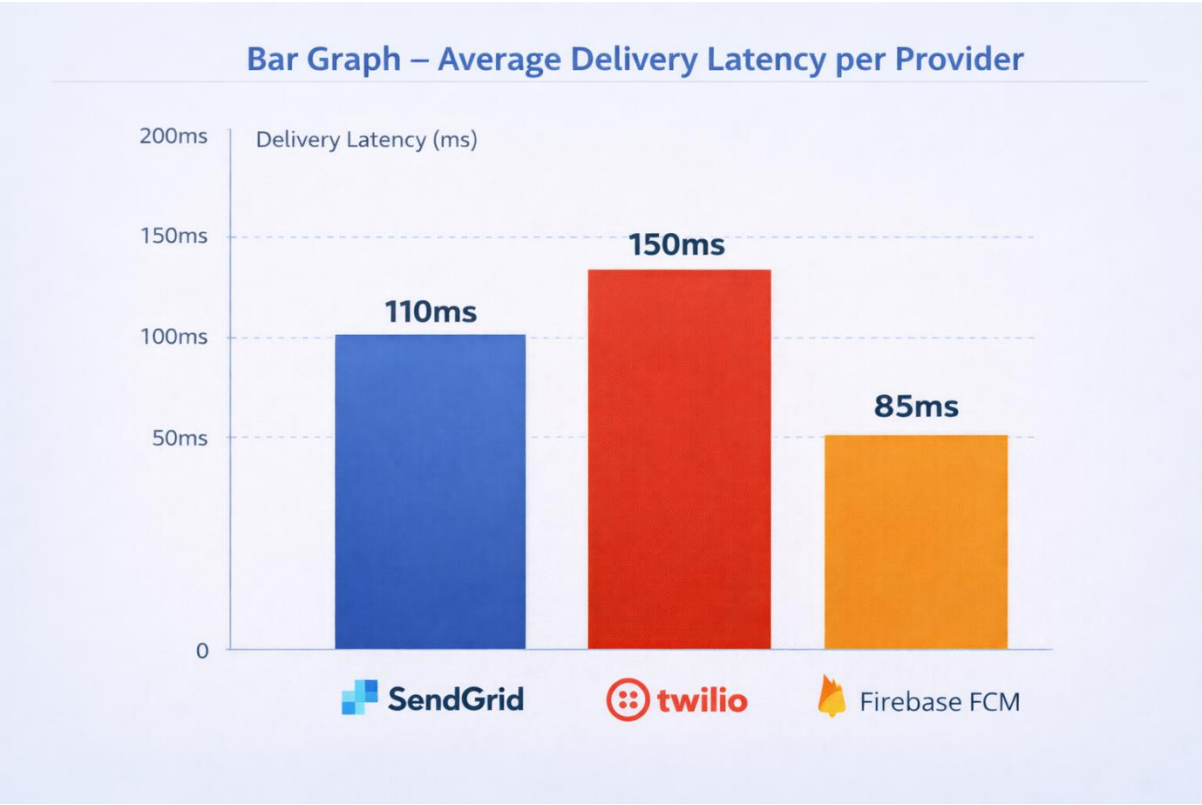
4.2 Template Management

Templates are stored in S3 buckets and versioned.

Template ID	Trigger Event	Channel	Retry Policy
tpl_welcome_01	User Signup	Email	3x
tpl_order_conf	Order Placed	Email + SMS	5x (Critical)
tpl_cart_aband	24h Inactivity	Push	0x (Best Effort)

See the full list of template variables in the Developer Portal.





4.3 External Dependencies

- **SendGrid API:** Used for transactional emails.
- **Twilio API:** Used for SMS OTPs.
- **Firebase FCM:** Used for mobile push notifications.